

DATABASE SYSTEMS BOOTCAMP

Practice Assignment

Transactions · Joins · Indexing · Query Optimization

Part 1 — Build Your Own Practice Schema

Before attempting the tasks below, design and create your own small database. Do not copy a schema from anywhere — write your own CREATE TABLE statements based on the description below, then insert your own sample data using INSERT statements.

Table	Columns	Notes
customers	id (primary key), name, email (unique), signup_date	Add at least 15–20 sample customers.
restaurants	id (primary key), name, cuisine	Add at least 8–10 sample restaurants.
orders	id (primary key), customer_id, restaurant_id, total, status, created_at	Add at least 40–50 sample orders. customer_id and restaurant_id must match real ids from your other two tables.
accounts	id (primary key), name, balance	Just 2 rows — e.g. one account named Alice and one named Bob — used only for Task 1.

Customers:

	id [PK] integer	name character varying (100)	email character varying (100)	signup_date date
1	1	Anand Kanzariya	anand@gmail.com	2026-07-01
2	2	Neesarg Sir	neesarg@gmail.com	2026-07-02
3	3	Dishant Sir	dishant@gmail.com	2026-07-02
4	4	Jayesh Sir	jayesh@gmail.com	2026-07-03
5	5	Jaydeep Dabhi	jaydip@gmail.com	2026-07-03
6	6	Gautam Parmar	gautam@gmail.com	2026-07-04
7	7	Dhruv Patel	dhruv@gmail.com	2026-07-04
8	8	Tushar Parmar	tushar@gmail.com	2026-07-05
9	9	Sneha Parmar	sneha@gmail.com	2026-07-05
10	10	Vivek Chauhan	vivek@gmail.com	2026-07-06
11	11	Komal Makwana	komal@gmail.com	2026-07-06
12	12	Nishil Rathod	nishil@gmail.com	2026-07-07
13	13	Savan Dabhi	savan@gmail.com	2026-07-08
14	14	Krunal Makwana	krunal@gmail.com	2026-07-08
15	15	Het Patel	het@gmail.com	2026-07-09

Restaurants:

	id [PK] integer	name character varying (100)	cuisine character varying (50)
1	1	Kathiyawadi King	Gujarati
2	2	Honest Restaurant	Fast Food
3	3	Milan Hotel	Gujarati
4	4	Rajwadi Hotel	Gujarati
5	5	Sankalp Restaurant	South Indian
6	6	Murlidhar Hotel	Fast Food
7	7	Punjabi Tadka	Punjabi
8	8	Cafe Coffee Day	Cafe
9	9	Umiyaji	Gujarati

Orders:

1	1	1	350.00	Completed	2026-07-01 10:15:00
2	2	2	420.50	Pending	2026-07-01 11:20:00
3	3	3	560.00	Completed	2026-07-01 12:10:00
4	4	4	280.00	Cancelled	2026-07-01 01:45:00
5	5	5	690.00	Completed	2026-07-02 09:30:00
6	6	6	480.00	Pending	2026-07-02 10:10:00
7	7	7	750.00	Completed	2026-07-02 11:40:00
8	8	8	320.00	Completed	2026-07-02 01:20:00
9	9	1	410.00	Pending	2026-07-03 09:10:00
10	10	2	520.00	Completed	2026-07-03 10:40:00
11	11	3	360.00	Cancelled	2026-07-03 12:00:00
12	12	4	610.00	Completed	2026-07-03 02:15:00
13	1	5	430.00	Completed	2026-07-04 09:20:00
14	2	6	295.00	Pending	2026-07-04 10:30:00
15	3	7	710.00	Completed	2026-07-04 11:50:00
16	4	8	390.00	Completed	2026-07-04 01:10:00
17	5	1	455.00	Pending	2026-07-05 09:45:00
18	6	2	620.00	Completed	2026-07-05 10:55:00
19	7	3	510.00	Cancelled	2026-07-05 12:25:00
20	8	4	340.00	Completed	2026-07-05 02:05:00

21	9	5	580.00	Completed	2026-07-06 09:35:00
22	10	6	460.00	Pending	2026-07-06 10:15:00
23	11	7	770.00	Completed	2026-07-06 11:45:00
24	12	8	310.00	Completed	2026-07-06 01:35:00
25	1	2	540.00	Completed	2026-07-07 09:25:00
26	2	3	365.00	Cancelled	2026-07-07 10:50:00
27	3	4	480.00	Completed	2026-07-07 12:10:00
28	4	5	695.00	Pending	2026-07-07 02:20:00
29	5	6	430.00	Completed	2026-07-08 09:40:00
30	6	7	820.00	Completed	2026-07-08 10:35:00
31	7	8	275.00	Pending	2026-07-08 11:55:00
32	8	1	515.00	Completed	2026-07-08 01:30:00
33	9	2	360.00	Completed	2026-07-09 09:20:00
34	10	3	610.00	Cancelled	2026-07-09 10:25:00
35	11	4	450.00	Completed	2026-07-09 11:40:00
36	12	5	590.00	Pending	2026-07-09 01:15:00
37	1	6	330.00	Completed	2026-07-09 03:10:00
38	5	7	780.00	Completed	2026-07-09 04:20:00
39	8	8	410.00	Pending	2026-07-09 05:10:00
40	10	1	550.00	Completed	2026-07-09 06:00:00

Accounts:

	id [PK] integer	name character varying (50)	balance numeric (10,2)
1	1	Alice	500.00
2	2	Bob	100.00

Part 2 — Tasks

TASK 1 Transactions (ACID)

Using your accounts table:

a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

Ans:

Query History

```

1 Update accounts
2 Set balance = balance - 200
3 Where name = 'Alice';
4 Update accounts
5 Set balance = balance + 200
6 Where name = 'Bob';
7 Rollback;
8
9
10 Select * From accounts;

```

Data Output Messages Notifications

Showing rows: 1 to 2 of 1 Page No: 1

id	name	balance
1	Alice	500.00
2	Bob	100.00

b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

Ans:

Query History

```

1 Update accounts
2 Set balance = balance - 200
3 Where name = 'Alice';
4 Update accounts
5 Set balance = balance + 200
6 Where name = 'Bob';
7
8 Commit;
9
10 Select * From accounts;

```

Data Output Messages Notifications

Showing rows: 1 to 2 of 1 Page No: 1

id	name	balance
1	Alice	300.00
2	Bob	300.00

c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

Ans: The ACID property that guarantees money is never lost during a transfer is Atomicity. Atomicity ensures that all steps of the transaction are completed successfully or none of them are applied. If any error occurs, the entire transaction is rolled back, so no money is lost or partially transferred.

TASK 2 Joins

Using your customers and orders tables:

a) Write an INNER JOIN that lists every customer who has placed at least one order.

Ans:

Query Query History

```

1 Select Distinct c.id, c.name, c.email
2 From customers c
3 Inner Join orders o
4 On c.id = o.customer_id;

```

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1 of 1

	id [PK] integer	name character varying (100)	email character varying (100)
1	12	Nishil Rathod	nishil@gmail.com
2	9	Sneha Parmar	sneha@gmail.com
3	4	Jayesh Sir	jayesh@gmail.com
4	3	Dishant Sir	dishant@gmail.com
5	10	Vivek Chauhan	vivek@gmail.com
6	8	Tushar Parmar	tushar@gmail.com
7	6	Gautam Parmar	gautam@gmail.com
8	2	Neesarg Sir	neesarg@gmail.com
9	11	Komal Makwana	komal@gmail.com
10	1	Anand Kanzariya	anand@gmail.com
11	7	Dhruv Patel	dhruv@gmail.com
12	5	Jaydeep Dabhi	jaydip@gmail.com

b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

Ans:

```

1 Select c.id,c.name,c.email
2 From customers c
3 Left Join orders o
4 On c.id = o.customer_id
5 Where o.customer_id Is Null;

```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	id [PK] integer	name character varying (100)	email character varying (100)
1	15	Het Patel	het@gmail.com
2	13	Savan Dabhi	savan@gmail.com
3	14	Krunal Makwana	krunal@gmail.com

c) In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).

Ans: The marketing team prefers query (b) because it includes customers who have never placed an order, allowing them to target those customers with promotional offers and encourage their first purchase.

TASK 3 Indexing

Using your orders table:

a) Pick a column that is not currently indexed (for example status or created_at).

Ans: Status

b) Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

Ans: Completed:

```
1 Explain Analyze
2 Select * From orders
3 Where status = 'Completed';
```

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loop Count
1	Seq Scan on orders	cost=0.00..18.25	rows=3	width=94	(actual time=0.011..0.014	rows=25.00	loop=...
2	Filter: ((status)::text = 'Completed')::text)						
3	Rows Removed by Filter:		15				
4	Buffers: shared hit=1						
5	Planning:						
6	Buffers: shared hit=30						
7	Planning Time:				0.340 ms		
8	Execution Time:				0.025 ms		

Cancelled:

```
1 Explain Analyze
2 Select * From orders
3 Where status = 'Cancelled';
```

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loop Count
1	Seq Scan on orders	cost=0.00..18.25	rows=3	width=94	(actual time=0.016..0.018	rows=5.00	loop=...
2	Filter: ((status)::text = 'Cancelled')::text)						
3	Rows Removed by Filter:		35				
4	Buffers: shared hit=1						
5	Planning Time:				0.070 ms		
6	Execution Time:				0.064 ms		

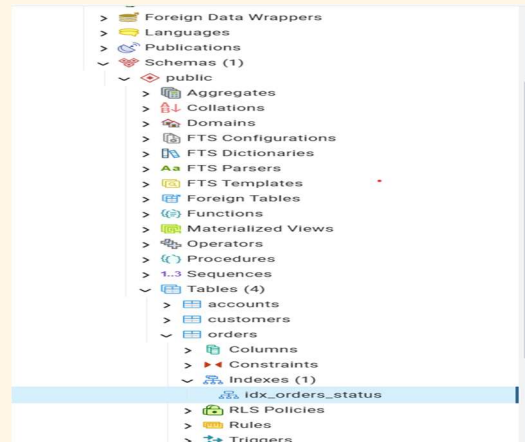
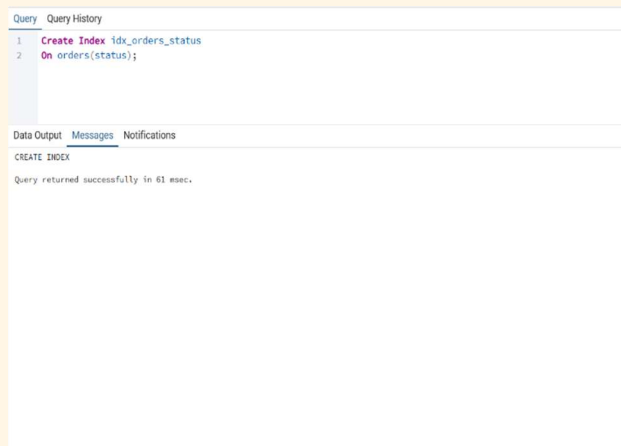
Pending:

```
1 Explain Analyze
2 Select * From orders
3 Where status = 'Pending';
```

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loop Count
1	Seq Scan on orders	cost=0.00..18.25	rows=3	width=94	(actual time=0.024..0.026	rows=10.00	loop=...
2	Filter: ((status)::text = 'Pending')::text)						
3	Rows Removed by Filter:		30				
4	Buffers: shared hit=1						
5	Planning Time:				0.088 ms		
6	Execution Time:				0.043 ms		

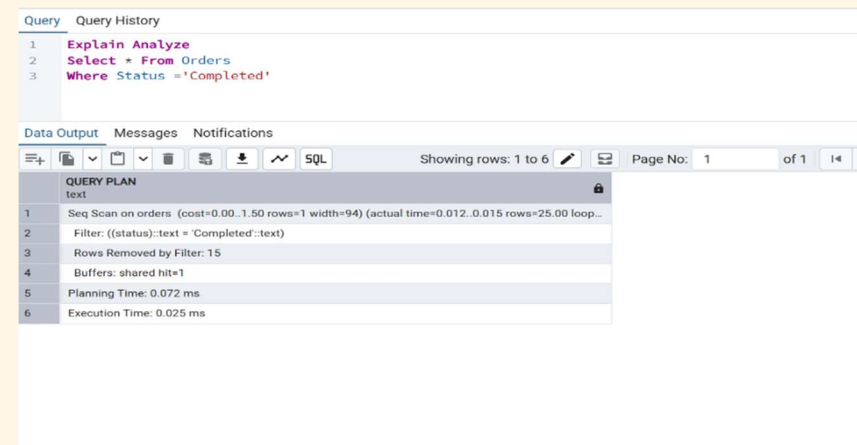
c) Create an index on that column.

Ans:

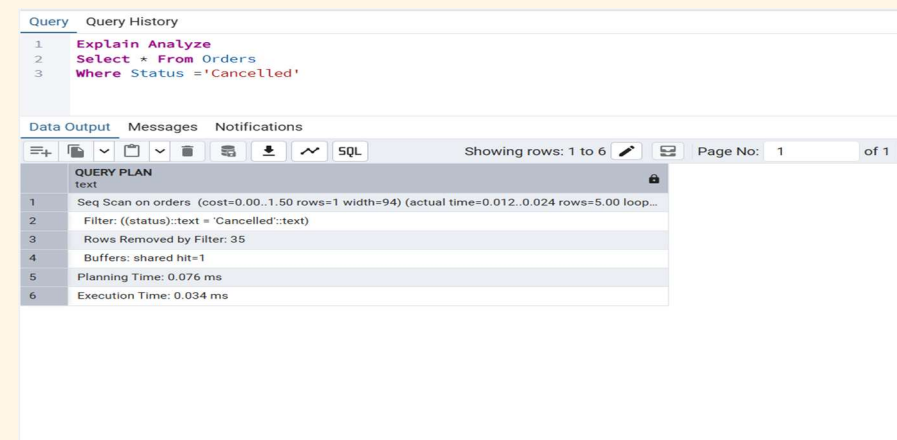


d) Re-run the exact same query and record the new numbers.

Ans: Completed:



Cancelled:



Pending:

The screenshot shows a PostgreSQL query editor interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying a SQL query:

```
1 Explain Analyze
2 Select * From Orders
3 Where Status ='Pending'
```

Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with 6 rows and 1 column. The table is titled 'QUERY PLAN' and has a lock icon. The rows contain the following information:

	QUERY PLAN
1	Seq Scan on orders (cost=0.00..1.50 rows=1 width=94) (actual time=0.016..0.018 rows=10.00 loop...
2	Filter: ((status)::text = 'Pending'::text)
3	Rows Removed by Filter: 30
4	Buffers: shared hit=1
5	Planning Time: 0.079 ms
6	Execution Time: 0.029 ms

At the bottom of the interface, there is a status bar showing 'Showing rows: 1 to 6' and 'Page No: 1 of 1'.

e) Submit a one-line comparison of before vs. after.

Ans: After creating the index, the query became slightly faster because PostgreSQL used the index to find the matching records instead of scanning the table. Since the dataset is small, the improvement was minimal.

TASK 4 Query Optimization

Start with this query, rewritten to use your own table and column names:

```
SELECT * FROM orders WHERE customer_id IN (
  SELECT id FROM customers WHERE signup_date >= 'some date'
);
```

a) Rewrite it as a JOIN instead of a subquery.

Ans:

Query		Query History	
1	Select o.*	From orders o	
2	Inner Join customers c	On o.customer_id = c.id	
3	Where c.signup_date >=	'2026-07-05';	
Data Output		Messages	
Notifications			
		Showing rows: 1 to 17	
		Page No: 1 of 1	
	id [PK] integer	customer_id integer	restaurant_id integer
	total numeric (10,2)	status character varying (20)	created_at timestamp without time zone
1	39	8	8
2	32	8	1
3	20	8	4
4	8	8	8
5	33	9	2
6	21	9	5
7	9	9	1
8	40	10	1
9	34	10	3
10	22	10	6
11	10	10	2
12	35	11	4
13	23	11	7
14	11	11	3
15	36	12	5
16	24	12	8
17	12	12	4
Total rows: 17		Query complete 00:00:00.088	

b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).

Ans:

Query		Query History	
1	Select o.id,c.name,o.total,o.status,o.created_at	From orders o	
2	Inner Join customers c	On o.customer_id = c.id	
3	Where c.signup_date >=	'2026-07-05';	
Data Output		Messages	
Notifications			
		Showing rows: 1 to 17	
		Page No:	
	id integer	name character varying (100)	total numeric (10,2)
	status character varying (20)	created_at timestamp without time zone	
1	39	Tushar Parmar	410.00
2	32	Tushar Parmar	515.00
3	20	Tushar Parmar	340.00
4	8	Tushar Parmar	320.00
5	33	Sneha Parmar	360.00
6	21	Sneha Parmar	580.00
7	9	Sneha Parmar	410.00
8	40	Vivek Chauhan	550.00
9	34	Vivek Chauhan	610.00
10	22	Vivek Chauhan	460.00
11	10	Vivek Chauhan	520.00
12	35	Komal Makwana	450.00
13	23	Komal Makwana	770.00
14	11	Komal Makwana	360.00
15	36	Nishil Rathod	590.00
16	24	Nishil Rathod	310.00
17	12	Nishil Rathod	610.00
Total rows: 17		Query complete 00:00:00.058	

c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

Ans: Original Query:

Query	Query History
1	Explain Analyze
2	Select * From orders
3	Where customer_id In (Select id From customers
4	Where signup_date >= '2025-07-05');

Data Output	Messages	Notifications
Showing rows: 1 to 13		
SQL		
QUERY PLAN		
1	Hash Join (cost=1.90..14.37 rows=13 width=94) (actual time=0.051..0.062 rows=40.00 loops=1)	
2	Hash Cond: (customers.id = orders.customer_id)	
3	Buffers: shared hit=2	
4	-> Seq Scan on customers (cost=0.00..12.12 rows=57 width=4) (actual time=0.022..0.025 rows=15.00 loo...	
5	Filter: (signup_date >= '2025-07-05'::date)	
6	Buffers: shared hit=1	
7	-> Hash (cost=1.40..1.40 rows=40 width=94) (actual time=0.022..0.023 rows=40.00 loops=1)	
8	Buckets: 1024 Batches: 1 Memory Usage: 11kB	
9	Buffers: shared hit=1	
10	-> Seq Scan on orders (cost=0.00..1.40 rows=40 width=94) (actual time=0.012..0.015 rows=40.00 loo...	
11	Buffers: shared hit=1	
12	Planning Time: 0.460 ms	
13	Execution Time: 0.090 ms	

Join Query:

Query	Query History
1	Explain Analyze
2	Select o.id,c.name,o.total,o.status,o.created_at From orders o
3	Inner Join customers c On o.customer_id = c.id
4	Where c.signup_date >= '2026-07-05';

Data Output	Messages	Notifications
Showing rows: 1 to 16		
SQL		
QUERY PLAN		
1	Hash Join (cost=1.90..14.37 rows=13 width=304) (actual time=0.043..0.046 rows=17.00 loops=1)	
2	Hash Cond: (c.id = o.customer_id)	
3	Buffers: shared hit=2	
4	-> Seq Scan on customers c (cost=0.00..12.12 rows=57 width=222) (actual time=0.017..0.018 rows=8.00 lo...	
5	Filter: (signup_date >= '2026-07-05'::date)	
6	Rows Removed by Filter: 7	
7	Buffers: shared hit=1	
8	-> Hash (cost=1.40..1.40 rows=40 width=90) (actual time=0.019..0.019 rows=40.00 loops=1)	
9	Buckets: 1024 Batches: 1 Memory Usage: 11kB	
10	Buffers: shared hit=1	
11	-> Seq Scan on orders o (cost=0.00..1.40 rows=40 width=90) (actual time=0.010..0.013 rows=40.00 loo...	
12	Buffers: shared hit=1	
13	Planning:	
14	Buffers: shared hit=172	
15	Planning Time: 1.291 ms	
16	Execution Time: 0.072 ms	

Total rows: 16	Query complete 00:00:00.068	CRLF	Ln 3, Col 2
----------------	-----------------------------	------	-------------

- Original Query (Subquery) → Execution Time = 0.090 ms
- Rewritten Query (JOIN) → Execution Time = 0.072 ms

The JOIN query executed slightly faster than the subquery because it directly combined the required tables. It also retrieved only the required columns, making the query more efficient.

Final Challenge — A Real-World Problem

Concepts used: this task combines everything from Tasks 2, 3, and 4 — Joins, Indexing, and Query Optimization — in one realistic scenario.

TASK 5 The Customer Profile Page

Scenario: a food-delivery company shows each customer their 5 most recent orders on their profile page — the order total, status, and the name of the restaurant it came from. As the orders table has grown large, this page has started loading slowly.

Complete the following steps, in order:

a) Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.

Ans: total Order for customer:

Query Query History

```

1 SELECT customer_id, COUNT(*) AS total_orders
2 FROM orders
3 GROUP BY customer_id
4 ORDER BY total_orders DESC;

```

Data Output Messages Notifications

Showing rows: 1 to 12 | Page No: 1 of 1

	customer_id integer	totalOrders bigint
1	5	4
2	10	4
3	1	4
4	8	4
5	12	3
6	4	3
7	2	3
8	11	3
9	3	3
10	9	3
11	7	3
12	6	3

```

1 SELECT c.name AS customer_name, r.name AS restaurant_name, o.total, o.status, o.created_at FROM orders o
2 JOIN customers c ON o.customer_id = c.id
3 JOIN restaurants r ON o.restaurant_id = r.id
4 WHERE o.customer_id = 1
5 ORDER BY o.created_at DESC

```

Data Output Messages Notifications

Showing rows: 1 to 4 | Page No: 1 of 1

	customer_name character varying (100)	restaurant_name character varying (100)	total numeric (10,2)	status character varying (20)	created_at timestamp without time zone
1	Anand Kanzariya	Murlidhar Hotel	330.00	Completed	2026-07-09 03:10:00
2	Anand Kanzariya	Honest Restaurant	540.00	Completed	2026-07-07 09:25:00
3	Anand Kanzariya	Sankalp Restaurant	430.00	Completed	2026-07-04 09:20:00
4	Anand Kanzariya	Kathiyawadi King	350.00	Completed	2026-07-01 10:15:00

b) Run EXPLAIN ANALYZE on it and record the rows examined and time taken.

Ans:

```

1  EXPLAIN ANALYZE
2  SELECT c.name AS customer_name, r.name AS restaurant_name, o.total, o.status, o.created_at FROM orders o
3  JOIN customers c ON o.customer_id = c.id
4  JOIN restaurants r ON o.restaurant_id = r.id
5  WHERE o.customer_id = 1
6  ORDER BY o.created_at DESC
7  LIMIT 5;

```

Data Output Messages Notifications

Showing rows: 1 to 24 Page No: 1 of 1

QUERY PLAN

text

```

1  Limit (cost=17.52, 17.92 rows=1 width=518) (actual time=0.044..0.045 rows=4.00 loops=1)
2    Buffers: shared hit=17
3    -> Sort (cost=17.52, 17.92 rows=1 width=518) (actual time=0.043..0.044 rows=4.00 loops=1)
4      Sort Key: o.created_at DESC
5      Sort Method: quicksort. Memory: 25kB
6      Buffers: shared hit=17
7      -> Nested Loop (cost=0.29, 17.91 rows=1 width=518) (actual time=0.030..0.038 rows=4.00 loops=1)
8        Buffers: shared hit=17
9        -> Nested Loop (cost=0.14, 9.67 rows=1 width=304) (actual time=0.027..0.032 rows=4.00 loops=1)
10          Buffers: shared hit=9
11          -> Seq Scan on orders o (cost=0.00, 1.50 rows=1 width=90) (actual time=0.015..0.017 rows=4.00 loops=1)
12            Filter: (customer_id = 1)
13            Rows Removed by Filter: 36
14            Buffers: shared hit=1

```

Total rows: 24 Query complete 00:00:00.072

15	-> Index Scan using customers_pkey on customers c (cost=0.14..8.16 rows=1 width=222) (actual time=0.003..0.003 rows=1.00 loops..)
16	Index Cond: (id = 1)
17	Index Searches: 4
18	Buffers: shared hit=8
19	-> Index Scan using restaurants_pkey on restaurants r (cost=0.14..8.16 rows=1 width=222) (actual time=0.001..0.001 rows=1.00 loops..)
20	Index Cond: (id = o.restaurant_id)
21	Index Searches: 4
22	Buffers: shared hit=8
23	Planning Time: 0.202 ms
24	Execution Time: 0.067 ms
Total rows: 24 Query complete 00:00:00.072	
	CRLF Ln 4

c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.

Ans:

```
Query    Query History
1  CREATE INDEX idx_customer_created
2  ON orders(customer_id, created_at);

Data Output: Messages Notifications
CREATE INDEX
Query returned successfully in 88 msec.
```

The screenshot shows the PostgreSQL Enterprise console interface. On the left, a tree view displays the database schema: MainData > Schemas > public > Tables > orders > Indexes > idx_orders_status. The 'idx_orders_status' index is selected and highlighted in blue. The right pane shows the SQL command: `CREATE INDEX ON orders(customer_id, c`. Below the command, a status message reads: "Data Output Messages Notification CREATE INDEX Query returned successfully in 88". The bottom status bar indicates the current context: "vvera > PostgreSQL > MainData > Schemas > public > Tables > orders > Indexes > idx_orders_status".

d) Re-run the same query and compare the new numbers to step (b).

Ans:

```
1  EXPLAIN ANALYZE
2  SELECT c.name AS customer_name,r.name AS restaurant_name,o.total,o.status,o.created_at FROM orders
3  JOIN customers c ON o.customer_id = c.id
4  JOIN restaurants r ON o.restaurant_id = r.id
5  WHERE o.customer_id = 1
6  ORDER BY o.created_at DESC
7  LIMIT 5;
```

Data Output Messages Notifications

Showing rows: 1 to 24 Page No: 1 of 1

QUERY PLAN

text

```
1  Limit (cost=17.92..17.92 rows=1 width=518) (actual time=0.045..0.046 rows=4.00 loops=1)
2    Buffers: shared hit=17
3    -> Sort (cost=17.92..17.92 rows=1 width=518) (actual time=0.044..0.045 rows=4.00 loops=1)
4      Sort Key: o.created_at DESC
5      Sort Method: quicksort Memory: 25kB
6      Buffers: shared hit=17
7      -> Nested Loop (cost=0.29..17.91 rows=1 width=518) (actual time=0.032..0.039 rows=4.00 loops=1)
8        Buffers: shared hit=17
9        -> Nested Loop (cost=0.14..9.67 rows=1 width=304) (actual time=0.028..0.033 rows=4.00 loops=1)
10          Buffers: shared hit=9
11          -> Seq Scan on orders o (cost=0.00..1.50 rows=1 width=90) (actual time=0.015..0.017 rows=4.00 loops=1)
12            Filter: (customer_id = 1)
13            Rows Removed by Filter: 36
14            Buffers: shared hit=1
```

15	-> Index Scan using customers_pkey on customers c (cost=0.14..8.16 rows=1 width=222) (actual time=0.003..0.003 rows=1.00 loops=1)
16	Index Cond: (id = 1)
17	Index Searches: 4
18	Buffers: shared hit=8
19	-> Index Scan using restaurants_pkey on restaurants r (cost=0.14..8.16 rows=1 width=222) (actual time=0.001..0.001 rows=1.00 loops=1)
20	Index Cond: (id = o.restaurant_id)
21	Index Searches: 4
22	Buffers: shared hit=8
23	Planning Time: 0.167 ms
24	Execution Time: 0.064 ms
Total rows: 24 Query complete 00:00:00.071 CRLF Ln 4, Col 1	

The execution time after creating the index was slightly lower than in step (b). The index helped the database find the required records more efficiently, although the improvement was small because the dataset is small.

e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.

Ans:

Query

Query History

1

Select r.name As restaurant_name,o.total,o.status,o.created_at From orders o

2

Join restaurants r On o.restaurant_id = r.id

3

Where o.customer_id = 1

4

Order By o.created_at Desc

5

Limit 5;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 4

Page No: 1 of 1

	restaurant_name character varying (100)	total numeric (10,2)	status character varying (20)	created_at timestamp without time zone
1	Murlidhar Hotel	330.00	Completed	2026-07-09 03:10:00
2	Honest Restaurant	540.00	Completed	2026-07-07 09:25:00
3	Sankalp Restaurant	430.00	Completed	2026-07-04 09:20:00
4	Kathiyawadi King	350.00	Completed	2026-07-01 10:15:00

f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

Ans: Before creating the index, the query took slightly more time to execute. After creating the index, the execution time improved slightly because the database located the required records more efficiently. Since the dataset is small, the improvement is minimal, but with a larger amount of data, the index would significantly improve query performance.

The index was created on customer_id and created_at because these columns are used in the WHERE and ORDER BY clauses. The index reduces the amount of data the database needs to search and sort, making the query more efficient. With a larger amount of data, the index provides much greater performance improvement because it avoids scanning the entire table.